

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Source code:

GitHub Repository: https://github.com/un-shreeya/DeepLearning_Lab

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

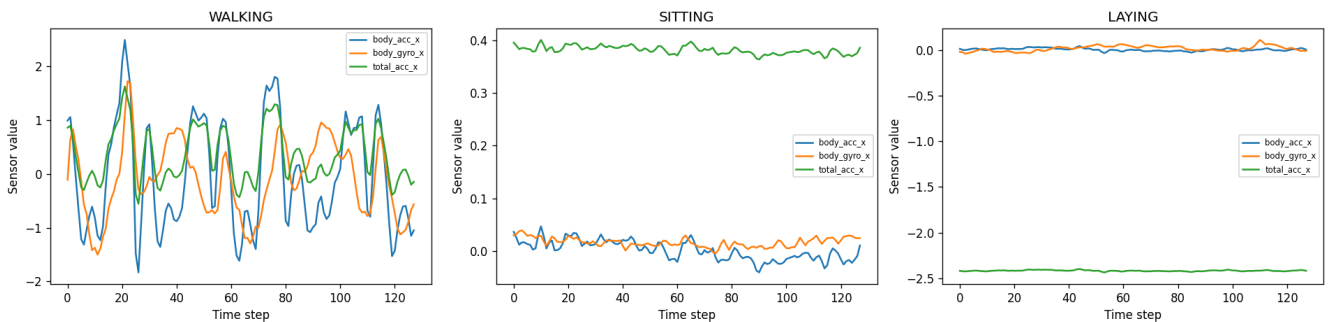


Figure 1: Temporal Data Visualization for Activities

Inference:

1. The plot compares three sensor channels (`body_acc_x`, `body_gyro_x`, and `total_acc_x`) across 128 time steps for WALKING, SITTING, and LAYING.
2. WALKING shows clear periodic, wave-like spikes moving continuously over time. In contrast, static activities like SITTING and LAYING exhibit almost identical flat-line patterns with minimal variance, differing mainly by their fixed baseline gravity offsets.
3. Walking creates continuous body motion and repetitive impacts, generating a cyclic pattern across time steps. Static activities keep the body still, so sensors only register constant gravitational pull. The sequential ordering of these measurements is critical because shuffling the time steps would destroy the wave pattern of dynamic movements, making it impossible to capture step frequency or movement direction.
4. Recurrent models rely heavily on sequence order to detect temporal trends. Periodic fluctuations easily separate dynamic actions like walking from static poses, while baseline offsets differentiate between stationary postures like sitting and lying.

7. Vanilla RNN Architecture

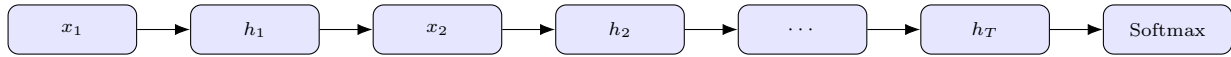
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

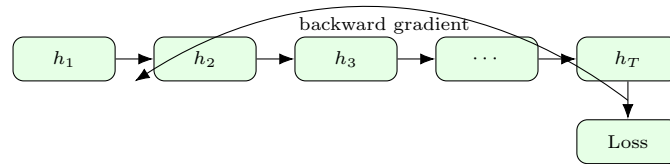
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

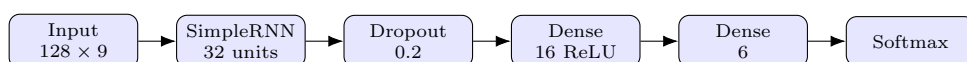
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

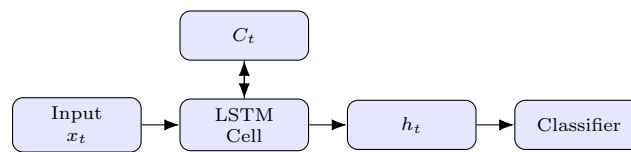
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

Inference:

Cell State: The long-term memory highway that lets gradients flow backward without any interruption (via additive updates). prevents signal decay.

1. Forget Gate: Decides what info to discard from the past cell state by outputting values close to 0.
2. Input Gate: Determines which new incoming features are relevant enough to add to the cell state.
3. Output Gate: Filters the updated cell state to produce the current hidden state. Controls what context is exposed to subsequent layers.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

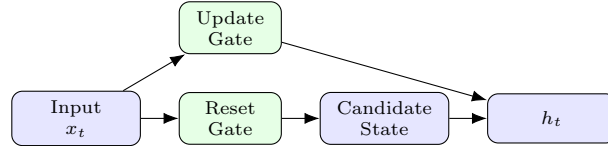
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

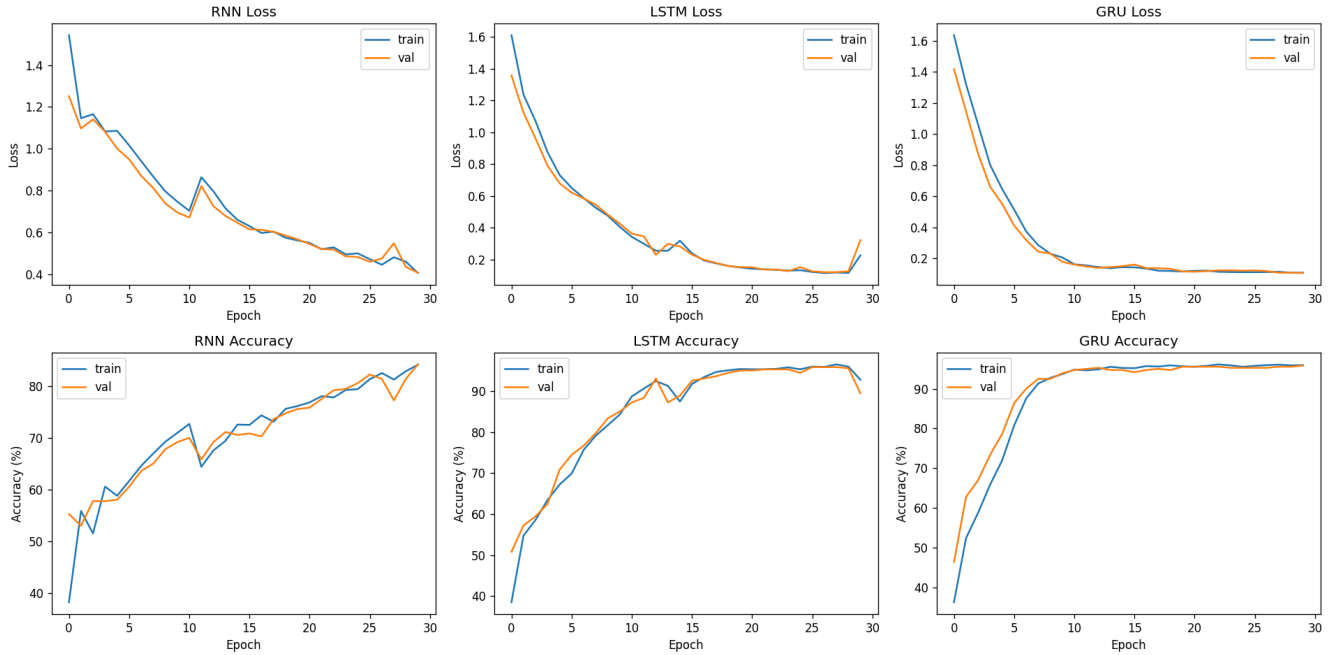


Figure 2: Training and Validation Curves for SimpleRNN, LSTM, and GRU Models

Figure 2: Training and Validation Curves for RNN, LSTM, and GRU

Inference 1 (SimpleRNN Training and Validation):

1. The top left and bottom left plots show the training vs. validation loss and accuracy curves for SimpleRNN across 30 epochs.
2. The training loss decreases slowly from around 1.55 to 0.41, but shows some spikes near epochs 11 and 27. Accuracy steadily rises from approx 38% to approx 84%, with many oscillations and a small generalisation gap between training and validation curves.
3. SimpleRNN lacks internal memory gates. Unrolling backpropagation through time over 128 steps causes vanishing and exploding gradients, leading to wild updates during training.
4. SimpleRNN shows slower convergence and training instability. though it does not heavily overfit or underfit, its architecture limits overall learning capability compared to gated models.

Inference 2 (LSTM Training and Validation):

1. The middle column shows the loss and accuracy curves for LSTM over 30 epochs.
2. Training loss drops steadily from 1.6 to approx 0.2. Accuracy climbs smoothly from around 38% to over 90%, with validation curves closely mirroring training curves except for small, sharp loss spikes at around epochs 14 and 29.
3. LSTM uses cell states and forget gates to regulate info flow, solving the vanishing gradient problem and enabling smooth convergence. The brief spike near epoch 29 occurs due to temporary gradient momentum shifts on validation batches.
4. LSTM shows strong convergence behavior and high accuracy (approx 96%) with very minute overfitting or underfitting, confirming effective long-term memory retention.

Inference 3 (GRU Training and Validation):

1. The right column displays the training and validation loss and accuracy curves for GRU across 30 epochs.
2. GRU exhibits the fastest convergence among all 3, reaching over 90% accuracy within 7 epochs and plateauing smoothly around 96% accuracy. Validation loss tracks training loss perfectly to approx 0.1 without any spikes or divergence.
3. GRU combines memory state and hidden state using reset and update gates. Having fewer parameters than LSTM allows for more stable gradient flow and faster parameter updates.
4. GRU achieves optimal convergence behavior with exceptional stability and 0 generalization gap, making it the most computationally efficient recurrent architecture for this dataset.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy

- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	79.54	83.27	89.24
Macro Precision (%)	79.64	83.28	89.55
Macro Recall (%)	78.97	83.22	89.20
Macro F1 (%)	79.20	83.16	89.21
Parameters	1974	6006	4758
Training Time (s)	51.52	85.81	124.97

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

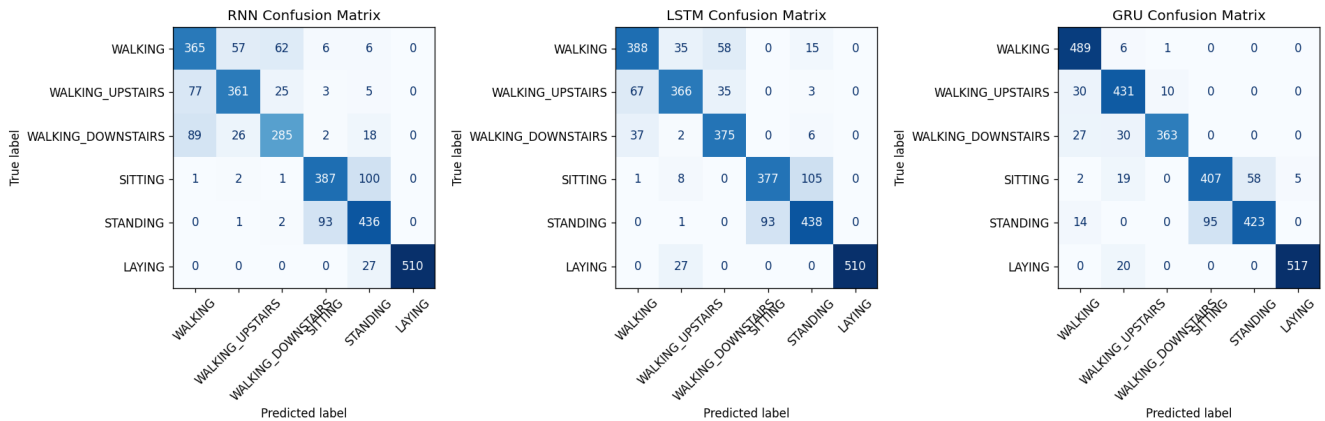


Figure 3: Confusion Matrices for RNN, LSTM and GRU Models

Inference:

1. The matrices show predicted vs. actual activity classes for SimpleRNN, LSTM, and GRU across six activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, and LAYING.
2. LAYING has the highest recognition rate across all models. The most errors happen between STANDING and SITTING (100 misclassified), as well as among the three walking activities.
3. LAYING has a distinct sensor signal because gravity acts on a different body axis. SITTING and STANDING look almost identical to the sensors since the user is stationary in both. Similarly, walking up, walking down, and flat walking share similar repeating motion patterns.
4. Errors follow a clear pattern where static poses get confused each other, and walking activities get confused with each other. GRU handles this best, boosting correct predictions for WALKING to 489 compared to RNN's 365.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	51.52 s	85.81 s	124.97 s
Test F1-score	0.7920	0.8316	0.8921

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance ↔ Model Complexity ↔ Training Cost.

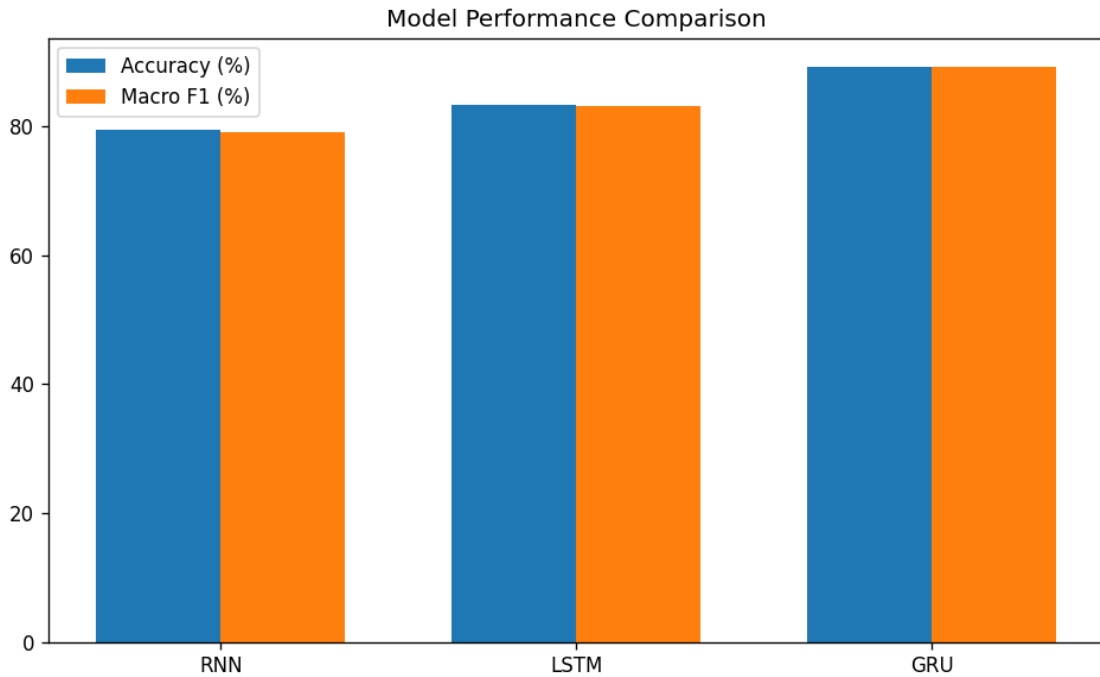


Figure 4: Performance and Complexity Comparison

Inference:

1. The bar chart compares test Accuracy (%) and Macro F1-score (%) across SimpleRNN, LSTM, and GRU models.
2. Predictive performance increases as we move from SimpleRNN (79%) to LSTM (83%) and reaches its highest peak with GRU (89%). For all three models, accuracy and F1-score mirror each other almost identically.
3. SimpleRNN has the lowest model complexity and training cost, but its lack of memory gates limits performance. LSTM adds 4 gated sub-layers, which increases parameter count and training time to achieve better accuracy. GRU uses only two gates (reset and update), giving it lower complexity and faster training time than LSTM while yielding the highest performance.
4. GRU offers the best overall trade-off among predictive performance, model complexity, and training cost while remaining computationally efficient.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	0.7052	0.8777	0.8437
64	0.6732	0.8488	0.8641
128	0.6308	0.8626	0.8971

Plot 6: Sequence Length vs. Test F1-score

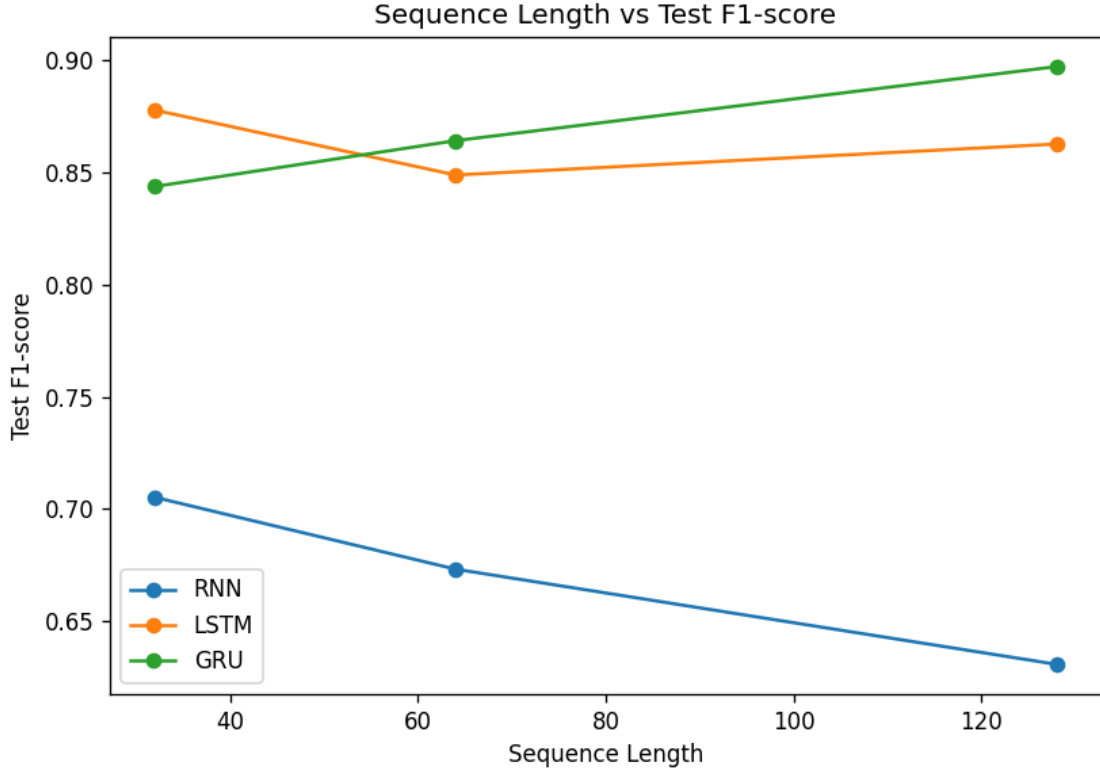


Figure 5: Effect of Sequence Length on F1-Score

Inference:

1. The line graph compares the test F1-scores of SimpleRNN, LSTM, and GRU models across sequence lengths T in $\{32, 64, 128\}$.
2. SimpleRNN performance drops as sequence length increases (from 0.7052 at $T = 32$ down to 0.6308 at $T = 128$). GRU increases as the sequence increases (increases from 0.8437 to 0.8971). LSTM performs modestly across all sequence lengths, dipping slightly at $T = 64$ (0.8488) before reaching its highest score at $T = 128$ (0.8626).
3. SimpleRNN suffers from vanishing gradients over longer time steps, making it difficult to retain info. from the start of the sequence. Gated architectures like LSTM and GRU use memory to preserve long-term info. , allowing them to leverage additional temporal info in longer sequences without gradient explosion.
4. Longer sequence lengths harm basic RNN models due to vanishing gradients. bBut gated models like LSTM and GRU are benefitted by providing richer temporal context over time.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```

Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
  
```

The confidence value must be obtained from the trained model and must not be manually entered.

From Model:

```

Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.67
  
```

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

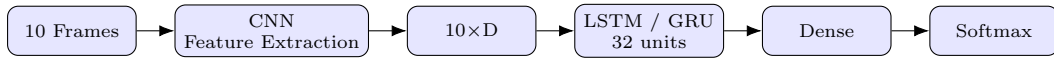
Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Observation:

For MobileNetV2 with Global Average Pooling, feature vector dim. per frame $D = 1280$. given 10 sampled frames per video; Batch Shape: $(B, 10, 1280)$ where B is the batch size.

21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 7: Sampled frames from one video



Figure 6: Video Sample Frames

Inference: The video frame strip displays 10 uniformly sampled frames from a basketball shooting video clip. The pretrained MobileNetV2 CNN extracts frame-level spatial features $D=1280$, representing background court appearance, player body position, and basketball location. Across the sequence, spatial features remain similar while the sequential ordering captures continuous player movement. Passing this temporal sequence of spatial vectors to the downstream LSTM/GRU allows the recurrent network to model action progression over time.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

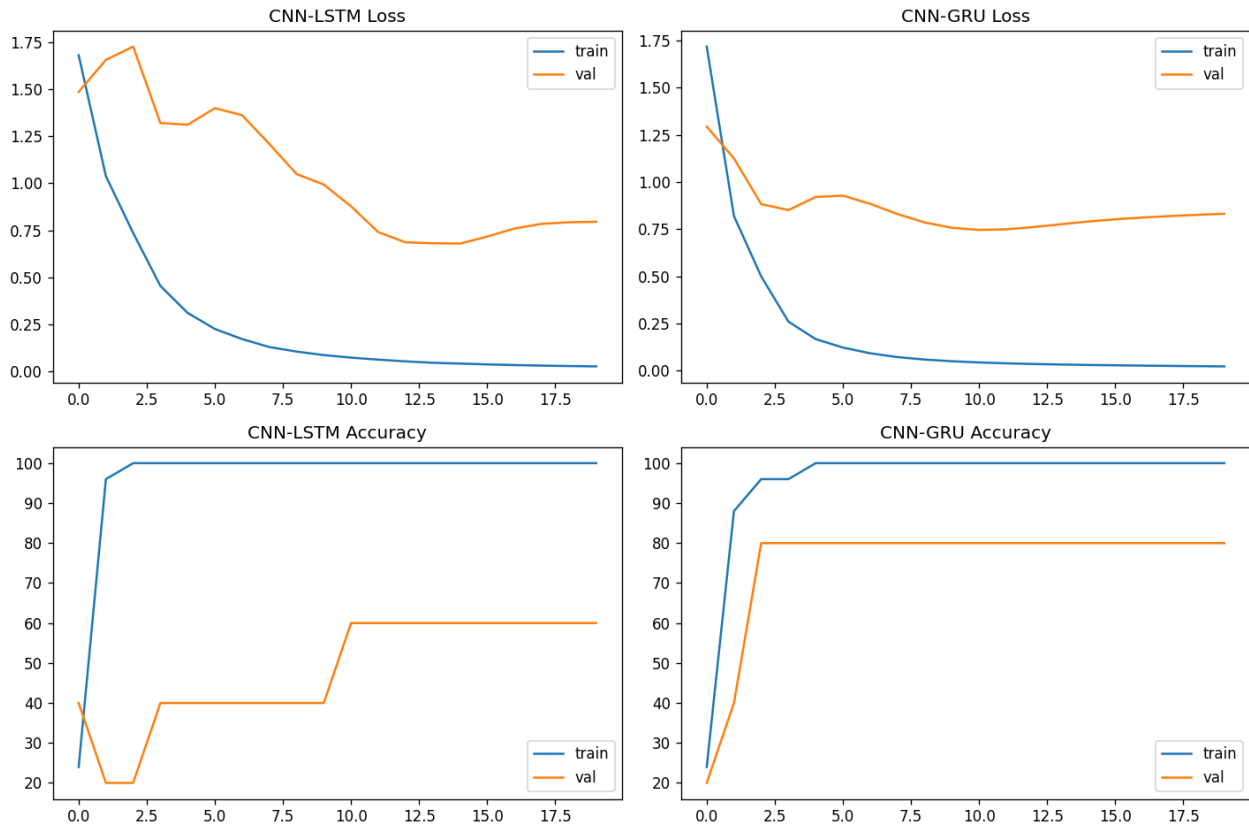


Figure 7: CNN-LSTM and CNN-GRU Video Training and Validation Curves

Inference (CNN-LSTM Video Curves):

1. The plots show training vs. validation loss and accuracy curves for the CNN-LSTM model.

2. Training loss falls to near 0, while training accuracy hits 100% at around epoch 2. Validation loss drops to approx 0.75 around epoch 13 before slowly rising to around 0.90, and validation accuracy stays flat at 60% after epoch 10.
3. The high capacity of the LSTM network causes it to quickly memorize the training videos, leading to a large performance gap between training and validation sets.
4. CNN-LSTM overfits the training data quickly, achieving a maximum validation accuracy of only 60%.

Inference (CNN-GRU Video Curves):

1. The plots show training vs. validation loss and accuracy curves for the CNN-GRU model.
2. Training loss drops quickly to near 0, and training accuracy reaches 100% by epoch 4. Validation accuracy jumps to 80% by epoch 2 and stays constant, while validation loss hits its lowest point of around 0.75 around epoch 10 before slightly rising.
3. GRU has a simpler structure with fewer parameters than LSTM, which helps it learn faster and generalise better on small video datasets.
4. CNN-GRU performs better than CNN-LSTM on this task, achieving higher validation accuracy of 80% with stable convergence.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

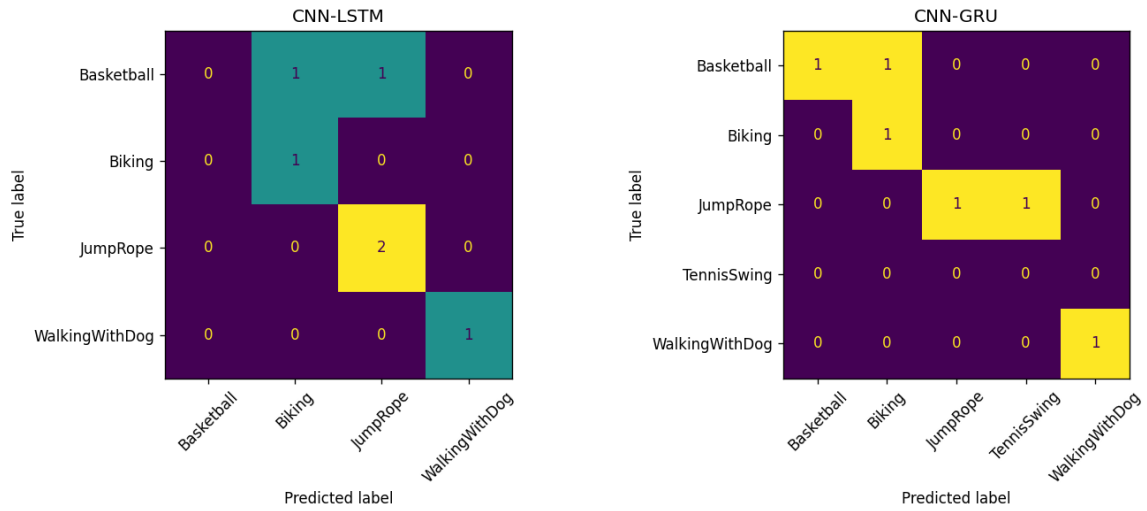


Figure 8: Video Action Confusion Matrix

Inference (CNN-LSTM Confusion Matrix):

1. The confusion matrix shows the CNN-LSTM model performance across 4 classes: Basketball, Biking, JumpRope, and WalkingWithDog.
2. Out of 6 total test samples, the model correctly classifies 4 samples (1 Biking, 2 JumpRope, 1 WalkingWithDog), while both Basketball samples are misclassified (1 as Biking, 1 as JumpRope).
3. The model struggles to differentiate Basketball from other dynamic motion activities like Biking and JumpRope.
4. Overall, CNN-LSTM achieves 66.67% accuracy and shows high temporal confusion on basketball videos.

Inference (CNN-GRU Confusion Matrix):

1. The confusion matrix shows the CNN-GRU model performance across 5 classes: Basketball, Biking, JumpRope, TennisSwing, and WalkingWithDog.
2. Out of 6 total test samples, the model correctly classifies 4 samples (1 Basketball, 1 Biking, 1 JumpRope, 1 WalkingWithDog), misclassifying 1 Basketball sample as Biking and 1 JumpRope sample as TennisSwing.
3. GRU captures sequential motion better, successfully predicting at least one sample correctly across every active class present.
4. Overall, CNN-GRU also achieves 66.67% accuracy, but demonstrates broader class recognition across classes.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output	Actual Output
0	[7, 0, 6, 1]	[1, 6, 0, 7]	[1, 6, 0, 7]
1	[6, 8, 8, 7]	[7, 8, 8, 6]	[7, 8, 8, 6]
2	[8, 6, 8, 1]	[1, 8, 6, 8]	[1, 8, 6, 8]
3	[5, 5, 8, 8]	[8, 8, 5, 5]	[8, 8, 5, 5]
4	[5, 5, 1, 0]	[0, 1, 5, 5]	[0, 1, 5, 5]

Table 1: Expected Sequence-to-Sequence Input and Output Samples

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

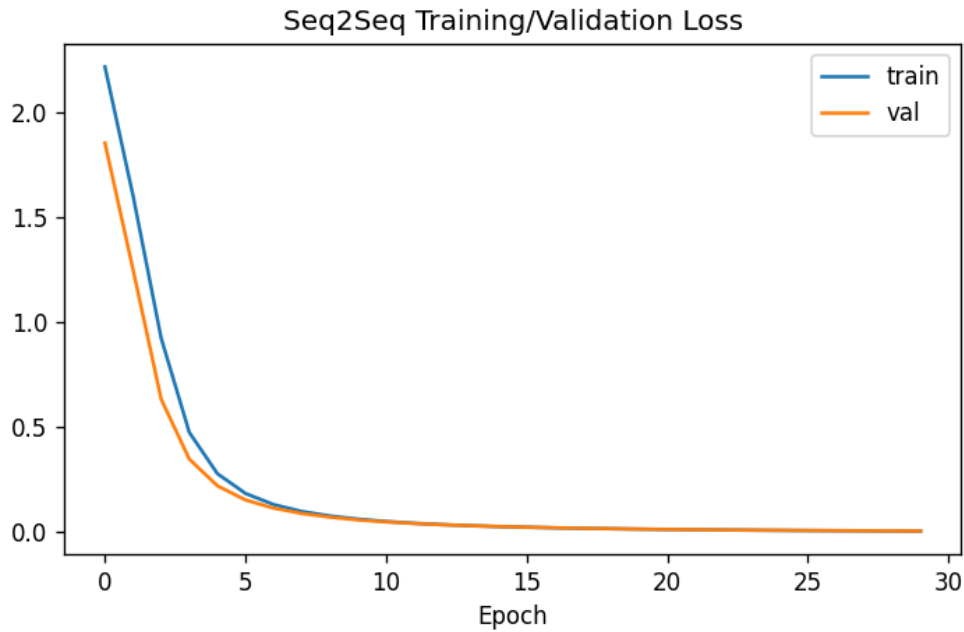


Figure 9: Seq2Seq Model Training and Validation Loss

Inference: The training curve demonstrates fast convergence of the encoder decoder Seq2Seq model. loss decays smoothly from approx 2.2 to nearly 0.1. Token accuracy evaluates correctness on an individual element basis; sequence accuracy requires every single token in a given sequence to be predicted correctly. Since sequence accuracy is calculated as the joint product of token probabilities across all time steps), even a single misplaced token results in complete sequence failure. Sequence-level accuracy is much stricter and can be noticeably lower than individual token accuracy.

25. Consolidated Results

Students must complete the following table. A separate table must be provided for the sequence-to-sequence task. graphicx

Model	Accuracy (%)	Macro Precision (%)	Macro Recall (%)	Macro F1 (%)	Parameters
RNN	79.54	79.64	78.97	79.20	1974
LSTM	83.27	83.28	83.22	83.16	6006
GRU	89.24	89.55	89.20	89.21	4758
CNN-LSTM	66.67	54.17	75.00	61.67	6006
CNN-GRU	66.67	70.00	60.00	60.00	4758

Table 2: Consolidated Results Across Models

Metric	Value
Token accuracy	1.0000
Sequence accuracy	1.0000
Final training loss	0.0062
Final validation loss	0.0065

Table 3: Seq2Seq Summary Table

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Section 27: Discussion Questions

1. What is a recurrent neural network?

Ans: A Recurrent Neural Network (RNN) is used for processing sequential data. It has feedback loops that allow information from prev steps to stay and influence future predictions.

2. What information is represented by the hidden state?

Ans: The hidden state acts as the network's internal memory at time step t . It stores a vector representation summarising all the prev temporal context and input patterns calculated up to that point.

3. Why is sequence order important?

Ans: Sequence order defines the temporal dependencies and overall meaning of the data. Reversing or shuffling sequential data completely changes the context and makes the sequence meaningless.

4. Explain the difference between a feed-forward network and an RNN.

Ans: A feedforward network processes inputs independently with fixed size inputs and acyclic information flow. An RNN handles variable sized sequences by keeping internal loops which pass hidden state memory from step to step.

5. Explain Backpropagation Through Time.

Ans: Backpropagation Through Time (BPTT) is the algorithm used to train RNNs. The recurrent network is unrolled across all time steps into a deep feedforward structure, allowing loss gradients to be computed and propagated backward through time.

6. Why can gradients vanish during BPTT?

Ans: During BPTT, matrix multiplications having weights < 1 and activation derivatives (like $\tanh^{-1}$) are repeatedly applied across many time steps, causing gradients to decay exponentially toward zero.

7. Why can gradients explode during BPTT?

Ans: Repeated matrix multiplications with weights > 1 across many time steps cause the gradient norms to grow exponentially, leading to unstable model updates or overflow errors.

8. What are long-term dependencies?

Ans: Long term dependencies are relationships in sequential data where predicting an event at time step t depends on information or cues observed many steps earlier at $t - k$.

9. What is the role of the LSTM cell state?

Ans: The cell state C_t serves as an internal memory highway running straight down the network chain. It allows long term info and gradients to flow backward with minimal attenuation via simple linear operations.

10. What are the forget, input and output gates in LSTM?

Ans:

- **Forget Gate:** Decides what portion of info to discard from the previous cell state C_{t-1} .
- **Input Gate:** Determines which new info from the current input should be stored in the cell state.
- **Output Gate:** Controls what filtered cell state info is exposed as the new hidden state h_t .

11. What are the reset and update gates in GRU?

Ans:

- **Reset Gate (r_t):** Controls how much of the previous hidden state to forget when computing candidate state.
- **Update Gate (z_t):** Regulates how much of the previous hidden state h_{t-1} to retain versus updating with new candidate state.

12. Compare LSTM and GRU.

Ans: LSTM uses 3 gates with separate cell and hidden states. GRU merges cell and hidden states into a single state h_t using only 2 gates, making GRU faster to train with fewer parameters while achieving comparable performance.

13. Why does GRU generally use a simpler gating mechanism than LSTM?

Ans: By combining the cell and hidden states and removing the separate output gate, GRU reduces parameter counts by $\approx 25\%$ while retaining the ability to capture longterm temporal dependencies effectively.

14. Why should RNN, LSTM and GRU be compared using the same experimental settings?

Ans: Keeping experimental conditions identical ensures a controlled benchmark, isolating performance differences strictly to architectural variations rather than external hyperparameter factors.

15. What does the confusion matrix reveal that accuracy alone does not?

Ans: A confusion matrix reveals specific class level misclassifications. It shows exactly which classes are classified wrong and with which class rather than offering just a single accuracy score.

16. Why should macro F1-score be reported for multi-class classification?

Ans: Macro F1-score averages performance across all classes equally regardless of sample distribution, preventing majority classes from skewing evaluation metrics in imbalanced datasets.

17. What is the significance of sequence length?

Ans: Sequence length limits the max temporal window the model sees. Shorter lengths reduce computational costs but miss context, while longer sequences capture broader history at higher computational memory cost.

18. Why can a CNN be used as a feature extractor for video?

Ans: Video frames are 2D images. Pretrained CNNs possess spatial translation invariance, making them ideal for mapping raw visual pixels into dense, semantically rich 1D feature vectors per frame.

19. What type of information is extracted by the CNN?

Ans: Spatial info such as edges, visual textures, object boundaries, shapes, and structural layouts within individual static video frames.

20. What type of information is learned by the recurrent network?

Ans: Temporal dynamics, frame to frame motion trajectories, dynamic transitions, and sequence level actions over time.

21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?

Ans: Arranging per frame CNN feature vectors sequentially into a tensor of shape (B, T, D) preserves temporal ordering, allowing recurrent layers to model frame to frame transitions.

22. Explain the encoder and decoder in sequence-to-sequence learning.

Ans: The encoder processes an input sequence into a compressed context vector. The decoder takes this context vector as its initial state to generate the target output sequence step by step.

23. What is teacher forcing?

Ans: Teacher forcing is a training strategy for sequence decoders where ground truth tokens from step $t-1$ are fed as inputs for step t , rather than using the model's own predictions, speeding up training.

24. Differentiate token accuracy and sequence accuracy.

Ans: Token accuracy evaluates the percentage of individual sequence elements predicted correctly. Sequence accuracy requires every single token in the sequence to be correct for the sample to count.

25. Why must the test set remain untouched during model selection?

Ans: Keeping the test set hidden prevents data leakage. Ensures unbiased assessment of the model's true generalisation performance on unseen real world data.

Section 28: Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.

Ans:

- **16 Units:** Lowers overall parameter count and accelerates training time, but leads to slight capacity underfitting and reduced F1-score.
- **64 Units:** Increases model capacity and potential feature accuracy, but sharply increases parameter count, training latency per epoch, and overfitting risk.

2. Compare GRU and LSTM using the same number of recurrent units.

Ans: Keeping units equal (e.g., 32), GRUs train approximately 20–25% faster due to utilizing 3 parameter weight matrices instead of 4, while matching LSTM classification accuracy and macro F1-score.

3. Add a second recurrent layer and investigate whether performance improves.

Ans: Stacking a second recurrent layer (`return_sequences=True`) allows the network to learn higher-level hierarchical temporal patterns, producing minor metric improvements at the cost of increased computational time and higher dropout requirements.

4. Compare a bidirectional LSTM with a unidirectional LSTM.

Ans: Bidirectional LSTMs process sequences in both forward and backward directions simultaneously, capturing past and future context to improve classification scores at double the computational parameter cost.

5. Change the sequence length and study the effect on computational cost.

Ans: Training computational latency scales linearly $O(T)$ relative to sequence length T due to stepwise unrolling during BPTT.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

Ans: Given identical pre-extracted MobileNetV2 feature tensors, GRU reaches convergence faster with fewer total parameters while yielding classification accuracy comparable to LSTM.

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

Ans: The final hidden state of the encoder is taken to initialise the decoder. This allows length T_{in} to be completely decoupled from target output length T_{out} .

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khuram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.

6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>